TurboRANSAC: A Fully GPU-Accelerated RANSAC Framework for Robust Estimation

Yifan Chen†, Feiyang Xin†, Zhen Yao, Shuhan Wu and Xin Lou
School of Information Science and Technology, ShanghaiTech University, Shanghai, China
Emails: {chenyf2022, xinfy2023, yaozhen2024, wushh12025, louxin}@shanghaitech.edu.cn

***Abstract*—Robust model estimation plays a central role in geometric computer vision, enabling applications such as camera localization, motion estimation, and 3D reconstruction. Among many techniques, Random Sample Consensus (RANSAC) is widely adopted for its robustness to outliers. However, existing RANSAC variants commonly rely on CPU-based sequential processing, leading to computational inefficiency that limits their suitability for real-time and large-scale scenarios. This challenge is heightened by the rise of learning-based pipelines optimized for GPU acceleration, where CPU-bound RANSAC becomes a bottleneck within predominantly GPU-centric workflows. In this paper, we propose TurboRANSAC, a fully GPU-accelerated framework that parallelizes all key stages, including sampling, hypothesis generation, scoring, and refinement, directly on the GPU. Unlike methods that optimize isolated components, TurboRANSAC provides a unified and scalable solution applicable to a broad spectrum of geometric estimation problems.**

***Index Terms*—RANSAC, GPU acceleration, Parallel processing, Robust estimation, Geometric computer vision.**

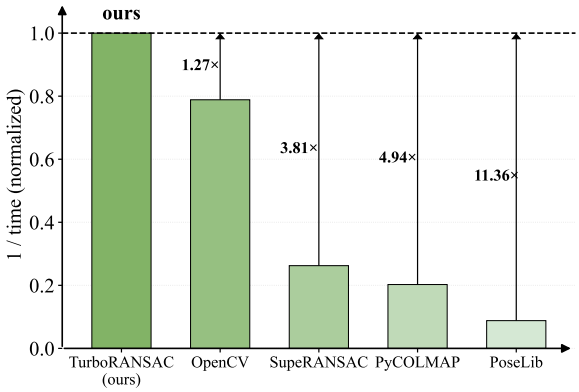


Fig. 1: Runtime comparison of different methods.

I. Introduction

Estimating geometric models in the presence of noise and outliers is a long-standing challenge in computer vision. A common solution, Random Sample Consensus (RANSAC) [1], has been widely adopted due to its robustness. It has become a cornerstone of many vision applications, including Structure-from-Motion [2], [3], epipolar geometry estimation [4], Homography fitting [5], [6], object pose estimation [7], [8], and Simultaneous Localization and Mapping (SLAM) [9]–[11], where RANSAC is frequently employed to robustly estimate models by repeatedly sampling minimal subsets, estimating models, and scoring them based on inlier counts.

Despite its widespread adoption, standard RANSAC suffers from several fundamental limitations. It becomes inefficient under low inlier ratios, unstable due to minimal sample degeneracy, and sensitive to manually tuned thresholds. Moreover, its inherently sequential nature restricts scalability and real-time applicability. These drawbacks have motivated decades of research focused on improving its efficiency, robustness, and generality. Sampling strategies such as PROSAC [12] and NeFSAC [13] enhance the likelihood of selecting correct inlier sets by leveraging prior knowledge or spatial coherence. Model verification methods, including MSAC, MLESAC [14], and MAGSAC++ [15], refine the scoring process such as LO-RANSAC [16] and Graph-Cut RANSAC [17] further improve model accuracy by iteratively refining the inlier sets. Several robust estimation frameworks—such as PoseLib [18], PyCOLMAP [19], and OpenCV [20]—have been developed to tackle geometric vision problems. However, they often struggle with scalability and limited parallel efficiency. More modular systems like SupeRANSAC [21] aim to integrate recent advances into flexible pipelines, yet most existing approaches remain CPU-bound and sequential processing. This makes them unsuitable for modern vision tasks demanding high-throughput and real-time performance, such as autonomous driving, Augmented Reality (AR), and Virtual Reality (VR).

Despite these advances, a unified and fully GPU-accelerated framework for robust estimation remains largely unexplored. To address this gap, we introduce TurboRANSAC, a framework designed to exploit massive parallelism for real-time geometric estimation. Our main contributions are as follows:

- A fully GPU-accelerated, CUDA library that unifies all RANSAC stages—including sampling, hypothesis generation, scoring, and refinement—into a single, high-performance framework for robust estimation.
- Proposed a ping-pong buffering strategy to merge batches across iterations, retaining the best model with reduced computational overhead and memory consumption.
- Achieves state-of-the-art speed while maintaining comparable accuracy and sub-linear runtime growth on large-

†These authors contributed equally to this work.

This work was supported by the Shanghai Disruptive Technology Innovation Program under grant 25DIPA00700

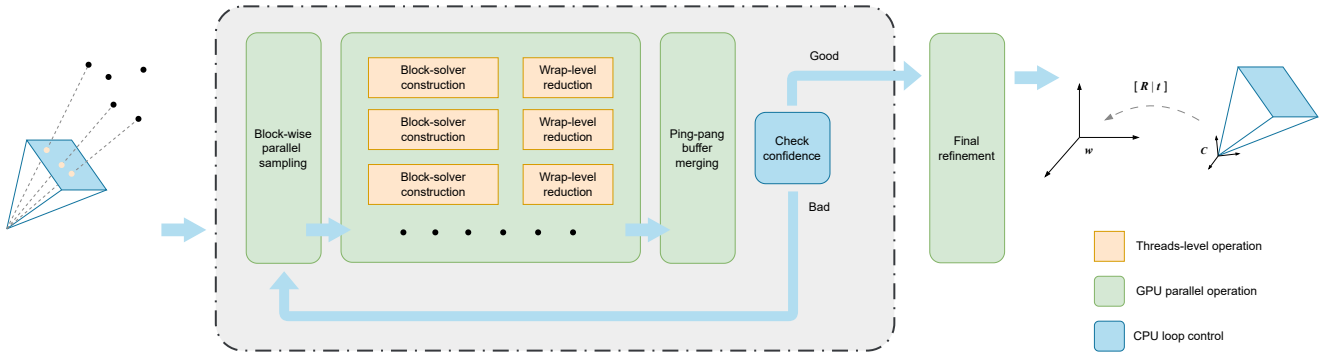


Fig. 2: The overall architecture of the proposed TurboRANSAC framework.

scale data.

II. Methodology

As illustrated in Fig. 2, this pipeline represents a typical absolute pose estimation task in TurboRANSAC framework, leveraging massive parallelism across all key stages, including: block-wise random sampling, block-wise hypothesis generation with warp-level reduction, pairwise merging under ping-pong buffering, iteration checking, and final refinement.

A. Preliminaries

To represent the general form of feature correspondences between two domains (e.g., 2D–2D or 2D–3D) in robust geometric estimation, we define the correspondence set as:

$$\mathcal{C} = \{\, (\mathbf{x}_i, \mathbf{y}_i) \mid i = 1, \ldots, N \,\}, \tag{1}$$

where each pair $(\mathbf{x}_i, \mathbf{y}_i)$ represents a matched observation between two spaces, with $\mathbf{x}_i \in \mathbb{R}^{d_x}$ and $\mathbf{y}_i \in \mathbb{R}^{d_y}$. For instance, in epipolar geometry, $d_x = 2$, $d_y = 2$ for 2D-2D correspondences, while in absolute pose estimation, $d_x = 2$ and $d_y = 3$ for 2D-3D correspondences. The goal is to estimate a robust model (e.g., a fundamental matrix or camera pose) that remains stable under noisy correspondences. In former CPU-based implementations, each correspondence pair is processed sequentially, thereby constraining computational efficiency.

We accelerate these computationally intensive tasks by leveraging NVIDIA GPUs with CUDA, where threads are organized into blocks scheduled on streaming multiprocessors (SMs). Within each block, threads are further grouped into warps of 32 threads that execute simultaneously on the hardware, enabling massive parallelism.

B. Block-wise random sampling

To efficiently generate minimal subsets for hypothesis testing in robust geometric estimation, we adopt a block-wise random sampling framework. For each CUDA block indexed by k, a minimal subset $\mathcal{S}_k \subset \mathcal{C}$ of size m is drawn (e.g., $m = 3$ for the P3P solver and $m = 8$ for the eight-point fundamental matrix solver) as:

$$\mathcal{S}_k = \{\, (\mathbf{x}_{s_i}, \mathbf{y}_{s_i}) \mid s_i = f_{\text{sample}}(i, N), \; i = 1, \ldots, m \,\}, \tag{2}$$

where $f_{\text{sample}}(\cdot)$ denotes the random index generation function. All sampled indices are enforced to be unique to avoid duplicate correspondences within each subset. Each block k initializes an independent random state derived from a global seed, the loop iteration, and its block index, enabling concurrent generation of multiple minimal subsets and uniform sampling over $\mathcal{C}$. Such block-wise initialization facilitates massive parallel exploration of the hypothesis space.

C. Block-wise Solver and Warp-level Reduction

Given a minimal subset $\mathcal{S}_k$, the task-specific solver generates a set of hypotheses:

$$\mathcal{H}_k = \{\mathcal{M}_k^{(1)}, \ldots, \mathcal{M}_k^{(j)}\}, \quad \mathcal{M}_k = f_{\text{solver}}(\mathcal{S}_k), \tag{3}$$

where $f_{\text{solver}}(\cdot)$ denotes the solver in block k. Each hypothesis is evaluated against the entire correspondence set $\mathcal{C}$ using a geometric loss function $\mathcal{L}(\cdot)$. For each correspondence $(\mathbf{x}_i, \mathbf{y}_i)$, the squared residual is computed as:

$$\mathcal{L}_i = \mathcal{L}(\mathbf{y}_i, g(\mathbf{x}_i; \mathcal{M}_k)), \tag{4}$$

where $g(\cdot)$ denotes the model prediction. Following the MSAC strategy, we employ a truncated quadratic loss to enhance robustness. In the TurboRANSAC framework, each CUDA thread is responsible for one correspondence; if the number of correspondences exceeds the threads per block, a stride loop ensures all correspondences are processed. The robust loss function for each correspondence is defined as

$$\rho(\mathcal{L}_i) = \min(\mathcal{L}_i^2, \tau^2), \tag{5}$$

where τ is the inlier threshold and $\mathcal{L}_i$ is the i-th of the The total MSAC cost for a hypothesis $\mathcal{M}_k$ is then computed using a warp-level (register-level) reduction:

$$E = \sum_{i=1}^{N} \rho(\mathcal{L}_i), \tag{6}$$

which minimizes global and shared memory accesses by storing partial results in local registers. Inliers are simultaneously identified using a bitmap representation:

$$\mathcal{I} = \{(\mathbf{x}_i, \mathbf{y}_i) \mid \mathcal{L}_i^2 < \tau^2\}. \tag{7}$$

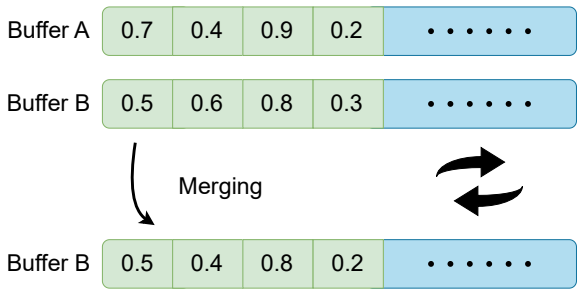


Fig. 3: Pairwise merge under ping-pong buffering.

Partial scores and inlier counts are subsequently aggregated hierarchically through warp-level reductions, yielding the block-wise total score:

$$E_{total} = E + (N - n)\tau^2, \tag{8}$$

where n is the number of inliers in the block. Finally, the best hypothesis within the block is selected as the one with minimal total cost:

$$\mathcal{M}_k^* = \arg \min_{\mathcal{M}_k \in \mathcal{H}_k} E_{total}. \tag{9}$$

These formulation maps directly to the CUDA kernel: each thread computes per-point losses in parallel, while hierarchical reduction produces the final MSAC score efficiently.

D. Pairwise Merge under Ping-Pong Buffering

To consolidate hypotheses generated across successive batches, we employ a pairwise reduction strategy using two alternating buffers, $\mathcal{B}^{(0)}$ and $\mathcal{B}^{(1)}$, each of size B. As illustrated in Fig. 3, the buffers alternate their roles between iterations to progressively merge and refine hypotheses. In iteration t, the active buffer index is determined by $b_t = t \bmod 2$, and each thread compares corresponding hypotheses from two buffers:

$$(\mathcal{M}_i^{(b_t)}, E_{total_i}^{(b_t)}) \leftarrow \begin{cases} (\mathcal{M}_i^{(b_t)}, E_{total_i}^{(b_t)}), & \text{if } E_{total_i}^{(b_t)} \le E_{total_i}^{(1-b_t)}, \\ (\mathcal{M}_i^{(1-b_t)}, E_{total_i}^{(1-b_t)}), & \text{otherwise.} \end{cases} \tag{10}$$

Here, $\mathcal{M}_i$ denotes the i-th hypothesis (e.g., pose parameters) in the batch and E_{total_i} its associated score. This operation performs an element-wise top-half selection across the two buffers, preserving only the superior hypothesis at each index. The updated buffer $\mathcal{B}^{(b_t)}$ thus holds the refined candidates. After each check iteration, the roles of the two buffers are interchanged, $\mathcal{B}^{(0)} \leftrightarrow \mathcal{B}^{(1)}$, effectively realizing the ping-pong buffering mechanism through pointer-level redirection rather than data movement. This alternating scheme sustains continuous GPU utilization and avoids redundant memory transfers or synchronization overhead.

Algorithm 1 The Proposed TurboRANSAC Framework

Require: Correspondences $\mathcal{C}$
Ensure: Refined best model $\widehat{\mathcal{M}}$

Initialize ping-pong buffers $\mathcal{B}^{(0)}, \mathcal{B}^{(1)}$ of size B
Set iteration index $t \leftarrow 0$
while $tB < K_{need}$ and $tB < K_{max}$ **do**
  Launch CUDA grid with B blocks, and t_b threads
  for all block b **do**
    Sample $\mathcal{S}_k \subset \mathcal{C}$ of m points
    $\mathcal{H}_k \leftarrow f_{\text{solver}}(\mathcal{S}_k)$
    for all thread t_b **do**
      Compute score E and inlier set $\mathcal{I}$
    end for
    $\mathcal{B}^{(b_t)} \leftarrow (\mathcal{M}_k^*, E_{total}, \mathcal{I})$
  end for
  Merge $\mathcal{B}^{(0)}$ and $\mathcal{B}^{(1)}$ into buffer $\mathcal{B}^{(b_t)}$
  Swap the buffer pointer $\mathcal{B}^{(0)} \leftrightarrow \mathcal{B}^{(1)}$
  if $E_{total_i}^{(b_t)} < E_{total}^*$ **then**
    $\mathcal{M}^* \leftarrow \mathcal{M}_i^{(b_t)}$, $E_{total}^* \leftarrow E_{total_i}^{(b_t)}$
  end if
  $K_{need} \leftarrow \frac{\log(1-p)}{\log(1-s^m)}$
  $t \leftarrow t + 1$
end while
return $\widehat{\mathcal{M}} \leftarrow \mathcal{R}(\mathcal{I}^*)$

E. Iteration Check and Final Refinement

To ensure that a correct hypothesis is obtained with probability at least p (confidence), the required number of iterations K is determined by the minimal sample size m and the inlier ratio s, as:

$$K = \frac{\log(1-p)}{\log\left(1 - s^m\right)}. \tag{11}$$

After hypothesis scoring, the most promising model $\mathcal{M}_k^*$ is further refined through a final re-estimation using all correspondences classified as inliers:

$$\widehat{\mathcal{M}} = \mathcal{R}\Big(\mathcal{I}\Big), \tag{12}$$

where $\mathcal{R}(\cdot)$ denotes a refinement operator, such as non-linear optimization. This refitting step improves the accuracy and stability of the final model, yielding smoother estimation results while preserving robustness to noise and outliers. Algorithm 1 illustrates the overall pipeline of the proposed framework.

III. Experiment

In this section, we conduct a comprehensive evaluation of our GPU-based RANSAC framework across different geometric estimation tasks, including fundamental matrix estimation and absolute pose estimation. The experiments are carried out on a publicly available dataset named 7-Scenes [22] to ensure diversity and comparability. We focus on measuring accuracy and runtime under varying outlier ratios and problem scales.

TABLE I
QUANTITATIVE COMPARISON ON THE 7-SCENES DATASET FOR FUNDAMENTAL MATRIX ESTIMATION.

Method	Chess	Fire	Heads	Pumpkin	Stairs	Avg. AUC	Time (ms)
OpenCV	0.91	0.44	0.94	0.58	0.94	0.76	1.90
SupeRANSAC	**1.07**	1.09	1.95	0.66	**1.15**	**1.19**	94.83
PoseLib	1.06	0.91	1.80	0.77	1.07	1.12	13.91
PyCOLMAP	0.92	**1.09**	1.83	**0.78**	0.91	1.11	6.11
TurboRANSAC	0.94	0.94	**1.96**	0.70	1.03	1.11	**0.77**

Each Cell Shows the AUC@10° Score (%), higher the better.

TABLE II
QUANTITATIVE COMPARISON ON THE 7-SCENES DATASET FOR ABSOLUTE POSE ESTIMATION.

Method	Fire	Heads	Office	Pumpkin	Time (ms)
OpenCV	0.25 / 0.64	0.35 / 0.58	0.29 / 0.95	0.31 / 1.26	20.10
PoseLib	0.23 / 0.61	0.31 / 0.53	0.27 / 0.89	0.29 / 1.17	180.03
SupeRANSAC	0.24 / 0.62	0.34 / 0.58	0.30 / 0.92	0.32 / 1.35	60.40
PyCOLMAP	0.23 / 0.61	0.30 / 0.51	0.26 / 0.89	0.28 / 1.12	78.25
TurboRANSAC	**0.23 / 0.61**	**0.30 / 0.51**	**0.26 / 0.89**	**0.28 / 0.98**	**15.85**

Each Cell Shows Rotation / Translation Error (° / cm), lower the better.

A. Dataset

We evaluate on the real-world 7-Scenes dataset [22], known for its challenging indoor sequences and camera poses, and a synthetic benchmark for scalability analysis. For consistency, 2D–2D correspondences (fundamental matrix) are generated using SuperPoint+LightGlue (SP+LG), while 2D–3D correspondences (absolute pose) use STDLoc. This ensures fair comparison across tasks using common matching strategies.

B. Implementation and Setup

a) Hardware: Our GPU method runs on an NVIDIA RTX 4090 (CUDA 11.5). CPU baselines are benchmarked on an Intel Core i7-14700K. Runtimes are reported per host; accuracy is directly comparable.

b) Experimental Settings: All methods adhere to shared RANSAC parameters for fairness, using established implementations for baselines (OpenCV, PyCOLMAP, PoseLib, SupeRANSAC). On 7-Scenes, we use a 1.0 px threshold and max 1000 iterations. On the synthetic benchmark (designed for scalability analysis with N up to 200k under a moderate-hard profile), we use 1.2 px, max 3000 iterations, and 0.9999 confidence for early stopping. No method-specific tuning is applied beyond these common settings. GPU timings reflect hot latency (kernel execution time, excluding CPU-GPU data transfers); this end-to-end overhead is negligible (e.g., ≈0.3 ms for N=200k) and does not alter the reported performance trends.

C. Results and Analysis

a) 7-Scenes Performance: On the real 7-Scenes dataset, TurboRANSAC demonstrates strong practical performance. For fundamental matrix estimation (Table I), it delivers highly competitive accuracy (Avg. AUC@10° 1.11), comparable to leading solvers like PyCOLMAP (1.11) and PoseLib (1.12). Notably, for absolute pose estimation (Table II), our method achieves **state-of-the-art accuracy** (Avg. 0.27° / 0.75 cm), matching or outperforming all competitors. Crucially, in both tasks, TurboRANSAC operates significantly faster, achieving average runtimes of just **0.77 ms** (fundamental matrix estimation) and **15.85 ms** (pose estimation), showcasing its efficiency

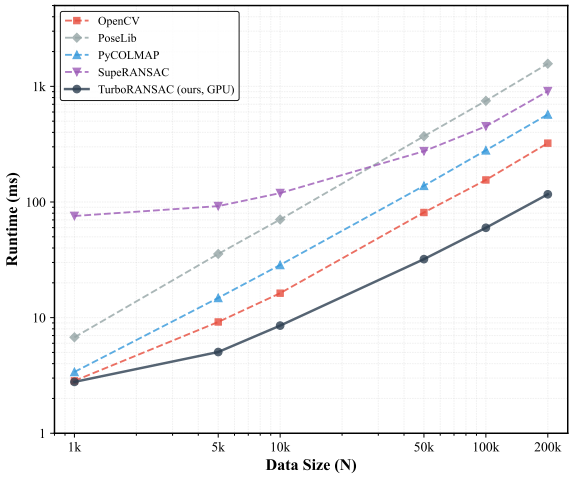


Fig. 4: Runtime for pose estimation on synthetic data.

b) Scalability on Synthetic Data: As shown in Fig. 4, TurboRANSAC's runtime grows sub-linearly as the number of correspondences N increases, contrasting sharply with the near-linear growth observed in CPU-based methods. Consequently, our approach achieves over 30× speedup at large scales (N = 200k) compared to methods like OpenCV and PyCOLMAP. Such favorable scaling ensures predictable performance, making TurboRANSAC ideal for real-time applications and pipelines involving modern dense feature matchers where traditional methods become bottlenecks.

IV. CONCLUSION

TurboRANSAC is a fully GPU-based framework for robust geometric model estimation. By parallelizing sampling, hypothesis generation, scoring, and refinement, it eliminates CPU bottlenecks and maximizes hardware utilization. The framework achieves substantial speedups while maintaining strong estimation accuracy across diverse benchmarks. A unified CUDA design with ping-pong buffering and shared-memory optimization minimizes synchronization overhead, while warp-level reduction further boosts computational efficiency. Together, these components enable practical real-time geometric estimation on modern GPUs. Future work will extend TurboRANSAC to homography estimation, 3D rigid pose estimation, multi-model fitting, and guided sampling strategies, broadening its applicability to large-scale 3D vision and real-time AR/VR scenarios.

REFERENCES

[1] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[2] L. Pan, D. Baráth, M. Pollefeys, and J. L. Schönberger, "Global structure-from-motion revisited," in *European Conference on Computer Vision*, pp. 58–77, Springer, 2024.

[3] P. Lindenberger, P.-E. Sarlin, V. Larsson, and M. Pollefeys, "Pixel-perfect structure-from-motion with featuremetric refinement," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5987–5997, 2021.

[4] V. Larsson, M. Pollefeys, and M. Oskarsson, "Orthographic-perspective epipolar geometry," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5570–5578, 2021.

[5] G. Nousias, K. Delibasis, and I. Maglogiannis, "H-RANSAC, an algorithmic variant for homography image transform from featureless point sets: Application to video-based football analytics," *arXiv preprint arXiv:2310.04912*, 2023.

[6] M. Rodríguez, G. Facciolo, and J.-M. Morel, "Robust homography estimation from local affine maps," *Image Processing on Line*, vol. 13, pp. 65–89, 2023.

[7] K. Guo, H. Ye, X. Gao, and H. Chen, "An accurate and robust method for absolute pose estimation with UAV using RANSAC," *Sensors*, vol. 22, no. 15, p. 5925, 2022.

[8] C. Riu, V. Nozick, and P. Monasse, "Automatic threshold RANSAC algorithms for pose estimation tasks," in *International Joint Conference on Computer Vision, Imaging and Computer Graphics*, pp. 1–20, Springer, 2022.

[9] H. Luo, Y. Liu, C. Guo, Z. Li, and W. Song, "SuperVINS: A real-time visual-inertial SLAM framework for challenging imaging conditions," *IEEE Sensors Journal*, 2025.

[10] S. Jung, W. Yang, and A. Kim, "Co-ral: Complementary radar-leg odometry with 4-dof optimization and rolling contact," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13289–13296, IEEE, 2024.

[11] S. Zhang, Y. Wang, J. Ma, W. Li, J. Yang, T. Yan, Y. Wang, L. Huang, M. Wang, and I. R. Atadjanov, "An error-matching exclusion method for accelerating visual SLAM," *arXiv preprint arXiv:2402.14345*, 2024.

[12] O. Chum and J. Matas, "Matching with PROSAC - progressive sample consensus," in *2005 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 220–226, 2005.

[13] L. Cavalli, M. Pollefeys, and D. Barath, "NeFSAC: Neurally filtered minimal samples," in *European Conference on Computer Vision*, pp. 351–366, Springer, 2022.

[14] P. H. Torr and A. Zisserman, "MLESAC: A new robust estimator with application to estimating image geometry," *Computer Vision and Image Understanding*, vol. 78, no. 1, pp. 138–156, 2000.

[15] D. Barath, J. Noskova, M. Ivashechkin, and J. Matas, "MAGSAC++, a fast, reliable and accurate robust estimator," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1304–1312, 2020.

[16] O. Chum, J. Matas, and J. Kittler, "Locally optimized RANSAC," in *Joint Pattern Recognition Symposium*, pp. 236–243, Springer, 2003.

[17] D. Barath and J. Matas, "Graph-cut RANSAC," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6733–6741, 2018.

[18] V. Larsson and contributors, "PoseLib - Minimal Solvers for Camera Pose Estimation," 2020.

[19] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *2016 IEEE/CVF Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4104–4113, 2016.

[20] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[21] D. Barath, "Superansac: One ransac to rule them all," *arXiv preprint arXiv:2506.04803*, 2025.

[22] J. Shotton, B. Glocker, C. Zach, S. Izadi, A. Criminisi, and A. Fitzgibbon, "Scene coordinate regression forests for camera relocalization in RGB-D images," in *2013 IEEE/CVF Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2930–2937, 2013.